

Space Engine Simulation: Object-Oriented Programming Laboratory Manual

This manual provides the architectural blueprint of the system. All explanations and requirements below directly align with the automated test suite your code must pass. Please read through the class descriptions, edge cases, and scenarios carefully.

System Architecture and Class Descriptions

1. Coordinate Class

This class represents a precise two-dimensional point in the universe.

- **Constructor:** Requires two parameters of type `double` representing the `x` and `y` coordinates.
- **Methods:**
 - `getX()` and `getY()`: Return the respective coordinate values as `double`.
 - `equals(Object o)`: Must be overridden to ensure logical equality. It must return `true` if and only if both `x` and `y` values are exactly the same. It must strictly return `false` if the compared object is `null`, if it belongs to a different class (like a `String`), or if only one of the coordinate axes matches.
 - `hashCode()`: Must be overridden so that two distinct `Coordinate` instances with identical `x` and `y` values produce the exact same hash code.
 - `toString()`: Must return a neatly formatted string strictly in the format: `(x, y)`.

2. CollisionException Class

A custom exception signaling that two space objects are attempting to occupy the exact same coordinate.

- **Inheritance:** Must directly extend the standard Java `Exception` class.
- **Constructor:** Takes a single `String` parameter for the error message.
- **Methods:** The exception must properly store this message so that the standard `getMessage()` method accurately retrieves the exact text provided during instantiation.

3. SpaceObject Class (Abstract)

The foundational blueprint for all celestial bodies. It ensures all space objects can be compared and sorted based on their location.

- **Inheritance & Interfaces:** Must be an abstract class and implement the `Comparable` interface to allow sorting against other `SpaceObject` instances.
- **Constructor:** Requires a `name` (`String`), `mass` (`double`), and `position` (`Coordinate`).
- **Methods:**
 - `getName()`, `getMass()`, `getPosition()`: Standard getter methods returning their respective data types.
 - `calculateVolume()`: An abstract method taking no parameters and returning a `double`.

- `compareTo(SpaceObject other)`: Compares objects based on their Euclidean distance from the origin point **(0, 0)**. Objects closer to the origin are considered "less than" objects further away.

4. Planet and Star Classes

Concrete subclasses that represent specific celestial bodies extending `SpaceObject`.

- **Constructor:** Both require `name` (`String`), `mass` (`double`), `position` (`Coordinate`), and an additional `radius` (`double`).
- **Methods:**
 - `calculateVolume()`: Implements the parent's abstract method using the plain text spherical volume formula: $V = (4.0 / 3.0) * \pi * r^3$.

5. PhysicsEngine Class

A utility class handling cosmic calculations. It must universally accept any combination of `SpaceObject` subclasses (e.g., a `Planet` and a `Star`, two `Planets`, etc.).

- **Methods:**
 - `calculateDistance`: Calculates the exact spatial distance between two objects using the Pythagorean theorem. Formula: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.
 - `calculateGravity`: Calculates the gravitational pull between two objects using Newton's law. Formula: $F = G * (m_1 * m_2) / d^2$, where the gravitational constant **G** is **6.674e-11**.

6. Universe Class

Acts as the main container, organizing space objects onto a spatial grid.

- **Constructor:** Initializes an empty internal collection to track objects by their `Coordinate`.
- **Methods:**
 - `addSpaceObject(SpaceObject obj)`: Places an object into the universe. If the specific `Coordinate` is already occupied by *any* type of object, it must throw a `CollisionException`.
 - `getObjectsSortedByDistance()`: Returns a `List` of all objects currently in the universe, strictly sorted from closest to the origin to furthest.

Critical Edge Cases (Tested Requirements)

Your implementation will be subjected to rigorous edge-case testing. Ensure your logic accounts for the following mathematical and structural rules:

- **Distance and Gravity Symmetry:** Calculations must be perfectly symmetric. The distance and gravity from Object A to Object B must exactly equal the distance and gravity from Object B to Object A.
- **Zero-Value Handling in Physics:**

- If the distance between two objects is exactly **0**, the `calculateGravity` method must safely return **0.0** to avoid division-by-zero errors.
- If either object has a mass of **0**, the gravitational force must naturally calculate to **0.0**.
- If a `Planet` or `Star` has a radius of **0**, its calculated volume must be exactly **0.0**.
- **Inverse Square Law Verification:** Your gravity calculation must properly apply the squared distance factor. If the distance between two objects doubles, the resulting gravitational force must drop to exactly one-quarter (**1/4**) of the original value.
- **Negative Coordinates and Quadrants:** Calculations and comparisons must work flawlessly across all Cartesian quadrants. Comparing an object at **(-3, 4)** and another at **(3, -4)** must yield a distance of **0** between their magnitudes (they are equally distant from the origin). Distance calculations between negative and positive coordinates (crossing the origin) must sum correctly.
- **Polymorphic Comparisons:** The system must seamlessly compare and calculate metrics between entirely different subclasses, such as checking if a `Star` is closer to the origin than a `Planet`.
- **Collision Persistence:** When a collision occurs and an exception is thrown, the original object occupying that space must remain perfectly intact and unaffected in the universe list.
- **Sorting Redundancies:** The sorting method must gracefully handle empty lists, already sorted lists, reverse-ordered lists, and multiple distinct objects that share the exact same distance to the center. All valid objects must be present in the final list without being overwritten by others at the same distance.

Example Scenarios

Scenario 1: Testing Equality and Hashing

The system creates a coordinate at **(5.5, -4.2)** and a second coordinate with identical values. It then places the first coordinate in a Hash-based map. If the overriding of `equals` and `hashCode` is correct, the system will successfully retrieve data using the second coordinate, recognizing them as the exact same location despite being different memory instances. Attempting to equate the coordinate with a `String` or `null` gracefully returns false.

Scenario 2: The Physics of Empty Space

Two identical objects are placed at the exact same coordinates (for a localized physics test, bypassing the Universe grid). Because their positions are identical, `calculateDistance` returns **0.0**. Consequently, `calculateGravity` detects this and safely returns **0.0** instead of crashing the program with an infinite value.

Scenario 3: Populating and Sorting the Cosmos

The universe is empty. You add the Sun at **(0, 0)**, Mars at **(4, 3)**, and a rogue comet at **(0, 5)**. The sorting algorithm evaluates their distances (Sun: 0, Mars: 5, Comet: 5). When retrieving the sorted list, the Sun

appears first. Mars and the Comet both appear next. Because they have the same distance magnitude, both are preserved in the list seamlessly without overwriting one another.

Scenario 4: Polymorphic Collisions

Earth (**Planet**) is placed at coordinate **(-5, 8)**. A new celestial body, Sirius (**Star**), is generated and inadvertently assigned the exact same coordinate **(-5, 8)**. When attempting to add Sirius to the Universe, the system recognizes the coordinate is taken. Even though one is a Planet and the other is a Star, the Universe strictly enforces spatial rules and throws a **CollisionException**, leaving only Earth safely at that coordinate.